2 UML Class Diagrams

The following complementary views describe the integrated Stage 3 implementation. They share class names across views; repeated boxes refer to the same classes. Figure 5 gathers the ownership, associations and state dependencies. We show key fields and operations and omit routine getters and constructors. +, – and # mean public, private and protected. Underlining marks static members. Fields marked `final` cannot be reassigned. A hollow triangle denotes inheritance, a filled diamond composition, a hollow diamond aggregation, an arrow association, and a dashed arrow dependency. Multiplicities describe one running game. State holders are classes of public static final String constants, not Java enums.

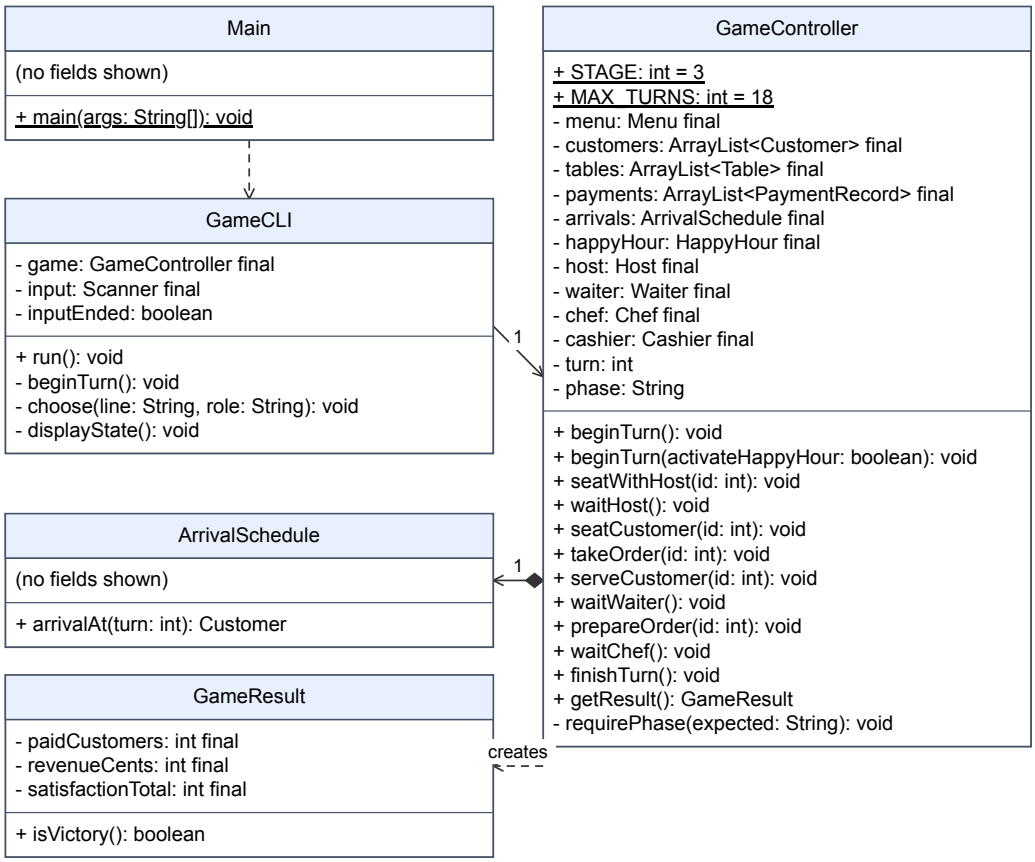


Figure 1. Boundary and turn control. Main constructs the CLI and controller; GameCLI requests actions and catches invalid choices. GameResult checks all three thresholds using unrounded payment totals.

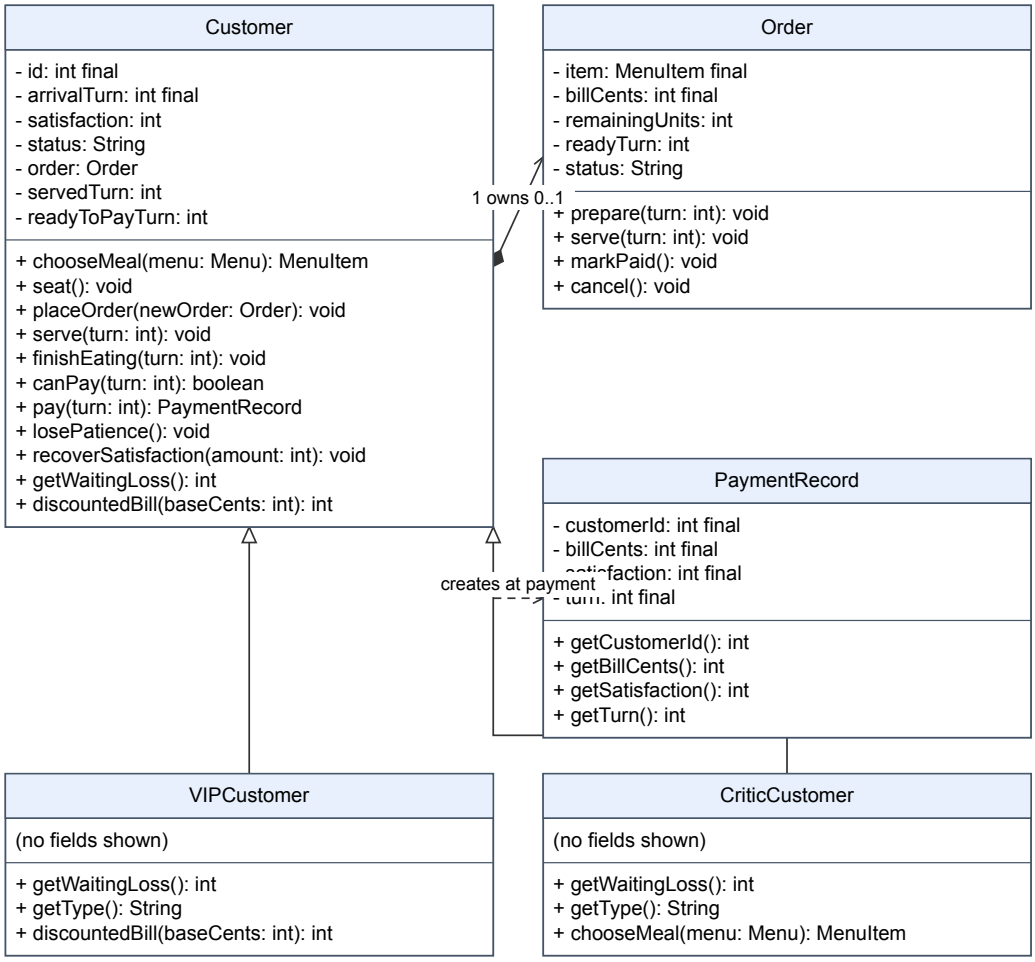


Figure 2. Customer and order lifecycle. Encapsulation: transition methods guard private state and timestamps. Composition: each Customer owns at most one Order. Polymorphism: Customer is the concrete Regular type; VIP and Critic override selected behaviour through Customer references. PaymentRecord preserves the bill and satisfaction at payment.

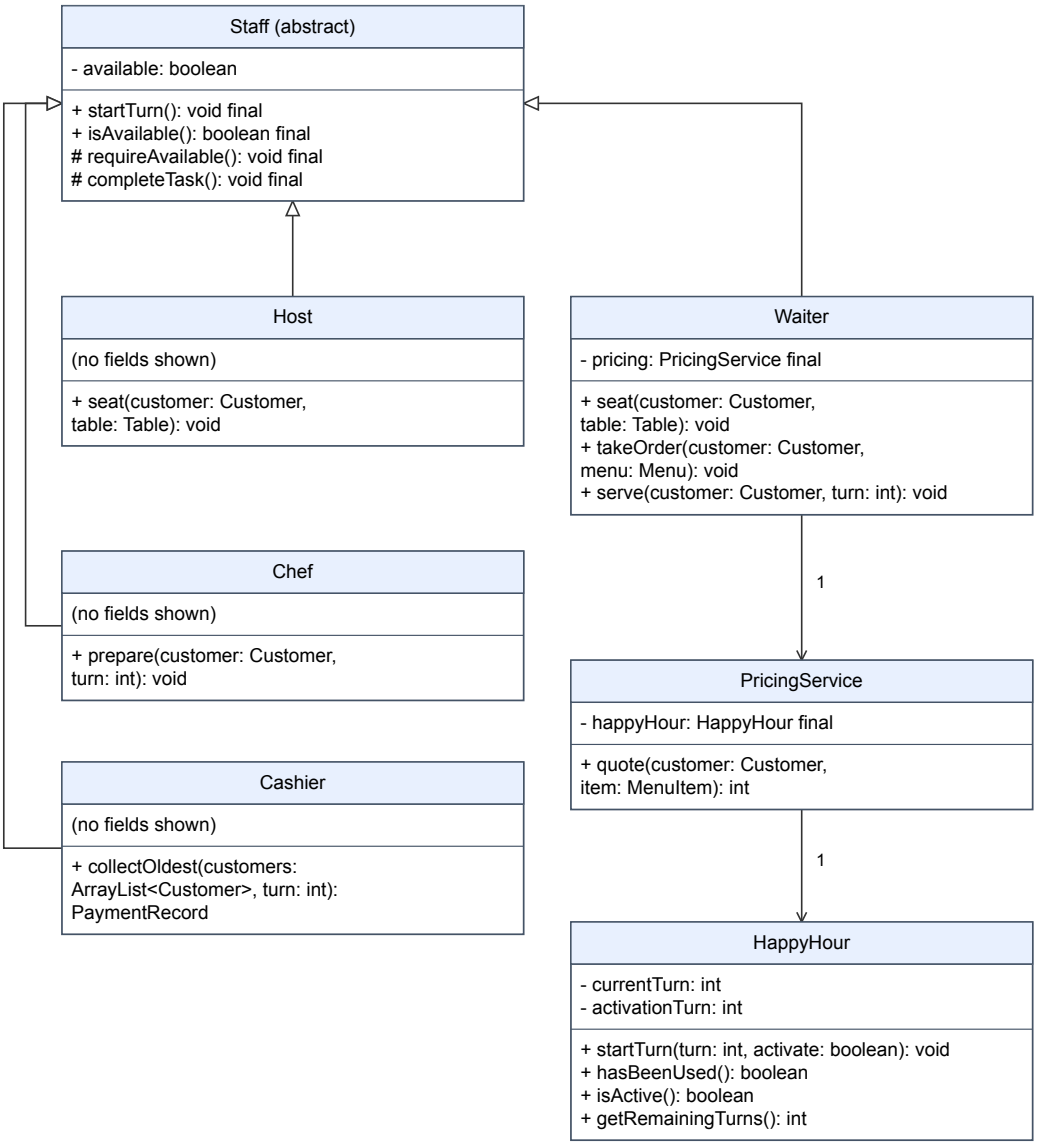


Figure 3. Staff and pricing. Staff shares one-task availability without declaring an unrelated generic work operation. Host and Waiter depend on Table; Chef depends on Order; Cashier depends on Customer and creates no payment until a customer is eligible. PricingService references the same HappyHour owned by the controller.

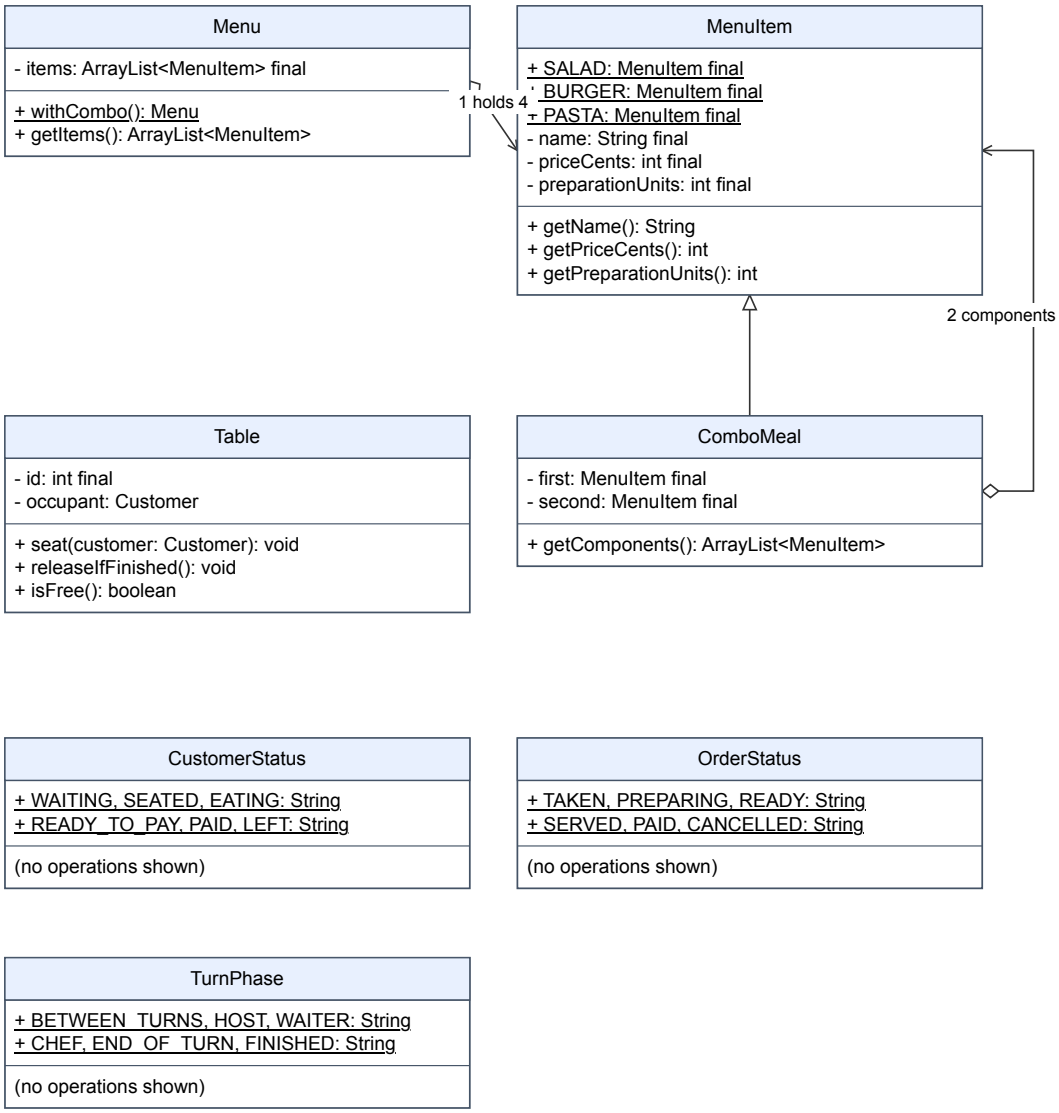


Figure 4. Menu, table and String constant holders. Menu items and combo components are shared immutable references, so hollow diamonds indicate aggregation. Each Table references 0..1 Customer; each Customer occupies 0..1 Table. An Order references exactly one MenuItem. Customer, Order and GameController depend on their respective status/phase constant classes. All constants shown are public static final; grouped names denote separate fields.

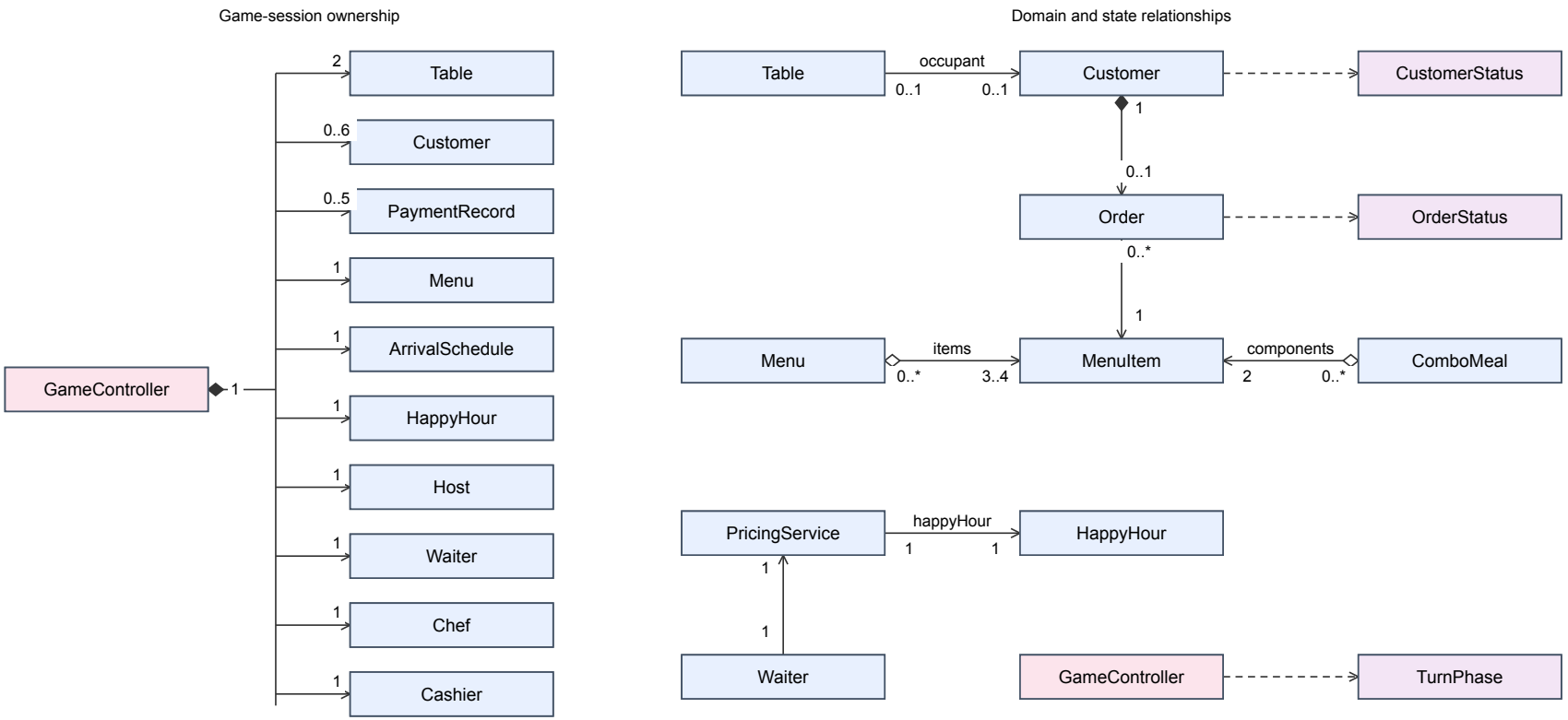


Figure 5. Relationship overview. The shared composition trunk on the left means that each listed object is owned by one GameController; numbers beside the destinations are per-game counts. On the right, endpoint numbers are association multiplicities. Menu and ComboMeal share immutable menu entries (hollow diamonds); Customer owns its Order (filled diamond). State dependencies point to String constant classes, not enums. Repeated names denote the same classes as in the detailed views.